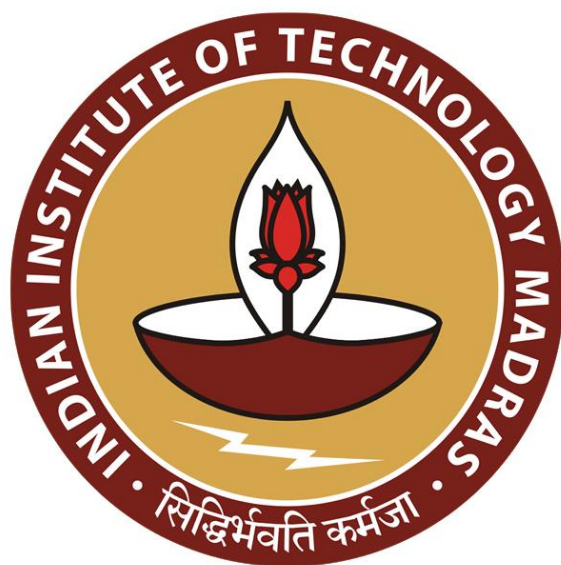




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Dealing with Errors

Of course, we would all like our programs to always be correct. But inevitably, there are mistakes or there are errors that occur when a program runs. And what we are going to talk about now is how to deal with these errors.

(Refer Slide Time: 00:26)

When things go wrong

- Our code could encounter many types of errors
 - *User input* — enter invalid filenames or URLs
 - *Device errors* — printer jam, network connection drops
 - *Resource limitations* — disk full
 - *Code errors* — invalid array index, key not present in hash table, refer to a variable that is `null`, divide by zero, ...
- Signalling errors
 - Return an invalid value: `-1` at end of file, `null`
 - What if there is no obvious invalid value?

Madhavan Mukund Dealing with errors Programming Concepts using Java 2/5

So, what kind of things could actually go wrong with your Java code? The most common thing is when your code expects input from a user, and the user does not follow the instructions or type something wrong. For instance, whenever you upload a file to a website, it will ask you to select the file name. Now, what if you mistyped the file, or when you go to the search bar of a browser and you type the URL of a website and you type it wrong, then what happens?

In this situation, the user input will produce a resource which the program cannot access and therefore they will naturally be an error. On the other side, you may be connected to something which is not working and your program has no way of detecting this, for instance, you may be trying to print to a printer where the paper has jammed or the paper has run out. Or you might be trying to connect to a website, but the internet connection has

stopped working for some reason. So, this is a different type of error that you could encounter.

Finally, you could also have a resource limitation, for instance, you might be trying to write something you might be trying to write a video to a disk, but your disk is full. So, these are all in some sense, external situations, which are outside the scope of your normal program behaviour. But nevertheless, if your program is interacting with an external environment, like a user, or with the internet, or with the disk or something, it should be aware of the fact that these kinds of errors can happen and perhaps take account of these errors when it executes.

Now, the fourth type of error is probably the most common in the sense that when we write our code, we make mistakes. Or, again it could be a mistake triggered by a user. But when we are accessing an array, for instance, we might ask for the i 'th index in an array, when i is actually out of bounds, it might be a negative number, or beyond the length of the array.

Or if we are trying to look up something in a hash table, the key that we produce is not a valid key. Or we might be trying to look up a variable whose value is not defined. So, we have an object which is actually null. Or in an arithmetic expression, some part of the expression is illegal, we might be trying to divide by 0 or take the square root of a negative number or something like this. So, these are errors that happen in our coding.

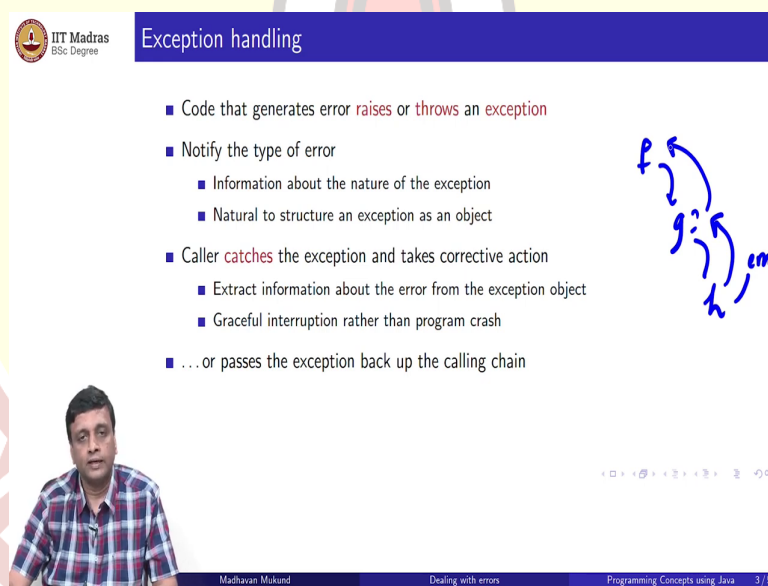
So, in any such situation, when we can anticipate what is going to happen, we would rather signal the error then let the code crash. If this error happens normally, and code is running, and nothing is there to stop it from crashing, it will crash. So, what we would like to do somehow intervene, anticipate these errors and take some preventive action. So, that means the code itself will have to signal somehow to the other parts of the code that an error has happened.

So, how can this happen normally in code? So, suppose you are trying to read something from a file, so a typical situation, which is not necessarily erroneous, but which is certainly an out of the ordinaries when you reach the end of the file. So, you are reading some data character by character from a file, you reach the end of the file, this is a special situation, it may not be an error, but it certainly is a special situation.

Now, you can try to return a value which is definitely not in the file. For instance, minus 1, write a character as it is encoded is always a positive number or a non-negative number. So, you cannot read a character from a file and get a negative number. So, if read function from a file returns a negative number, perhaps you know that there has been an error. Or if you are trying to call a function, which creates a node for instance. And if the node cannot be created, for whatever reason, if the function returns a null instead of a valid object, then maybe you know that the function failed to execute.

So, these are ways in which you can signal that something went wrong by returning some value which is outside the expected range. But this is not always possible of course. So, there may be some situations where there is no obvious value that is invalid. So, there is no way to signal by the value that you return to the user of the function or the code that what has happened is not quite normal.

(Refer Slide Time: 04:39)



Exception handling

- Code that generates error **raises** or **throws** an **exception**
- Notify the type of error
 - Information about the nature of the exception
 - Natural to structure an exception as an object
- Caller **catches** the exception and takes corrective action
 - Extract information about the error from the exception object
 - Graceful interruption rather than program crash
- ... or passes the exception back up the calling chain

Handwritten notes: f, g, h, em with arrows indicating flow.

Madhavan Mukund | Dealing with errors | Programming Concepts using Java | 3 / 5

- Code that generates error **raises** or **throws** an **exception**
- Notify the type of error
 - Information about the nature of the exception
 - Natural to structure an exception as an object
- Caller **catches** the exception and takes corrective action
 - Extract information about the error from the exception object
 - Graceful interruption rather than program crash
- ...or passes the exception back up the calling chain
- Declare if a method can throw an exception
 - Compiler can check whether calling code has made a provision to handle the exception

$f \rightarrow j()$



Madhavan Mukund

Dealing with errors

Programming Concepts using Java 3/5

So, this brings us to the topic of what we are going to talk about at length this week, which is exception handling. So, exception in this case, talks about an exceptional situation, a situation which is out of the ordinary. So, when your code encounters a situation which is out of the ordinary, it might be an error, or it might just be an unusual situation like reaching the end of the file, then it was signal it somehow.

So, in the language of exception handling, what we say is that you raise or throw an exception. So, the word raise and throw is used in common programming language terminology. Individual programming languages use one or the other. For example, if you have paid attention in Python, then in Python, you raise exceptions, what we will see is that in Java, you throw exceptions, so the actual word is not important, but the important thing is like a referee in a football match.

If you have watched a football match, there will be these linesmen running up and down. And when somebody is offside, they will raise a flag, or they might throw a flag. So, this is really the kind of situation, something unusual happens. So, the referee makes a point of drawing attention to it by raising or throwing something. So, that is exactly the origin of these words, raising and throwing exceptions.

Of course, what we had just seen before is that when you return a minus 1, for instance, it is very clear that you are trying to talk about an end of file exception. But there is a kind of encoding in that, the person calling the function should realize that if you get a minus 1 back, it means end of file.

Or if you are trying to create some kind of a node in a linked list, and you get a null back that for whatever reason, the node could not be created, but you do not know what reason. So, you would like to be more explicit in that. So, you would actually like to not only raise or throw an exception, but also provide information about the type of error. So, the nature of the exception.

So, in an object-oriented setup, this is a natural way to do this, which is that the exception itself, what you give, so the flag that the referee throws, is actually an object, it has structure. And inside that structure, you can store information about the nature of the error, or the exception that you have discovered.

So, the advantage of this structured object is that at the other end, somebody receives this object. So, in a standard way, they can inquire from the object, what went wrong. So, the object might record a string, at the very basic level, saying that this was the nature of the exception described in words. It could also be for instance, if a value was wrong, it could also tell you what the wrong value was, supposing you were expecting only positive values, and sometimes a negative value came, it might tell you that the negative value that showed up was minus 7. So, this might help you to diagnose what went wrong.

So, from the caller's point of view, when this exception object comes back in instead of the normal return value, the caller can now inspect this exception value and do something based on this exception value rather than abort the program. So, you can gracefully interrupt your execution and take some corrective action and proceed rather than letting the program crash.

Now, you are not obliged to do this. There may be no sensible way for this function, which calls the other function to take any action. So, then if you do not do anything, by default, this value gets passed back to the function that called you. So, there is a kind of chain of calls. So, f calls g, g calls h.

And now supposing here, there is an error. So, h will notify g, where g does not know what to do with it. So, by default, it will just go back to f. But now, of course, if f does not know what to do with it, and f for say, main, then what happens is that it will come out and the program will crash.

Now, one of the things that we need to know in this is when we call, when f calls g. In order to make meaningful use of the fact that g has diagnostic information, f should know that g calls, may throw an error of a particular type. So, we would also like to document that fact. So, we should also check when we call a function whether or not it, so just like we know what the arguments are and what the return type is, this is another type of return value.

When I call a function, I know what arguments I am supposed to pass so that is in the header of the function, I know what value is supposed to return in terms of the type is supposed to give me back an or whatever. Now, I should also know what kind of error is going to give me back if at all.

So, we can declare if a method throws an exception. And the advantage of this is now that a compiler can now check this. So, compiler can check that if g is known to throw exceptions and f is calling g, then has f taken appropriate steps to ensure that when it calls g it can handle the exceptions that come back. So, this is an extra bonus step that we will insist on which need not be done. So, a language like Python does not have this but we will see the Java does enforce this.

(Refer Slide Time: 09:41)

IIT Madras
BSc Degree

Java's classification of errors

- All exceptions descend from class Throwable
- Two branches, Error and Exception

```
graph TD; Throwable --> Error; Throwable --> Exception
```

Madhavan Mukund Dealing with errors Programming Concepts using Java 4/5

- All exceptions descend from class `Throwable`
 - Two branches, `Error` and `Exception`
- `Error` — relatively rare, “not the programmer’s fault”
 - Internal errors, resource limitations within Java runtime
 - No realistic corrective action possible, notify caller and terminate gracefully



Madhavan Mukund

Dealing with errors

Programming Concepts using Java 4/5

So, in Java, as you would expect, we said that we will organize our exceptions as objects. So, we will throw these errors and exceptions around as objects with structure. So, since they are objects, they must belong to classes which fit into this Java class hierarchy, the one which, the capital object is the root of the hierarchy. So, in Java, everything that is of this form, some kind of an error comes from a class called `Throwable`.

So, `Throwable` is the parent class of all types of errors and exceptions, and under `Throwable`, so `Throwable` is there and under `Throwable`, you have to subtypes, sub classes rather `Error` and `Exception`. So, the names are not very meaningful, they are just some historical names that Java chose. So, `Throwable` signifies that all of these things can be thrown when you find an error.

Now, the name error and exception is kind of up to taste, but what Java means by an error is really something that loosely speaking is not the programmers fault. So, this could be a problem with some resources inside the Java Virtual Machine, which is executing the code. So, there is some resource limitation, which has happened. And it is not the programmer's fault, because the programmer was not aware of the resources and was not checking, is not expected to check these limitations.

So, an internal error in the JVM would be classified in Java as belonging to this class error. So, in such situations, there is nothing much that you can do because the JVM has basically run out of some resource. So, all you can do is if you detect such a thing, if you want to

detect such a thing, all you can do is notify the caller that such an error happened, and just give up, you know terminate gracefully.

(Refer Slide Time: 11:26)

Java's classification of errors

- All exceptions descend from class **Throwable**
 - Two branches, **Error** and **Exception**
- **Error** — relatively rare, “not the programmer’s fault”
 - Internal errors, resource limitations within Java runtime
 - No realistic corrective action possible, notify caller and terminate gracefully
- **Exception** — two sub branches
 - **RunTimeException**, **checked exceptions**

Handwritten notes on the first slide:
A tree diagram showing Throwable branching into Error and Exception, with Exception further branching into RTE (RunTimeException) and checked exceptions.

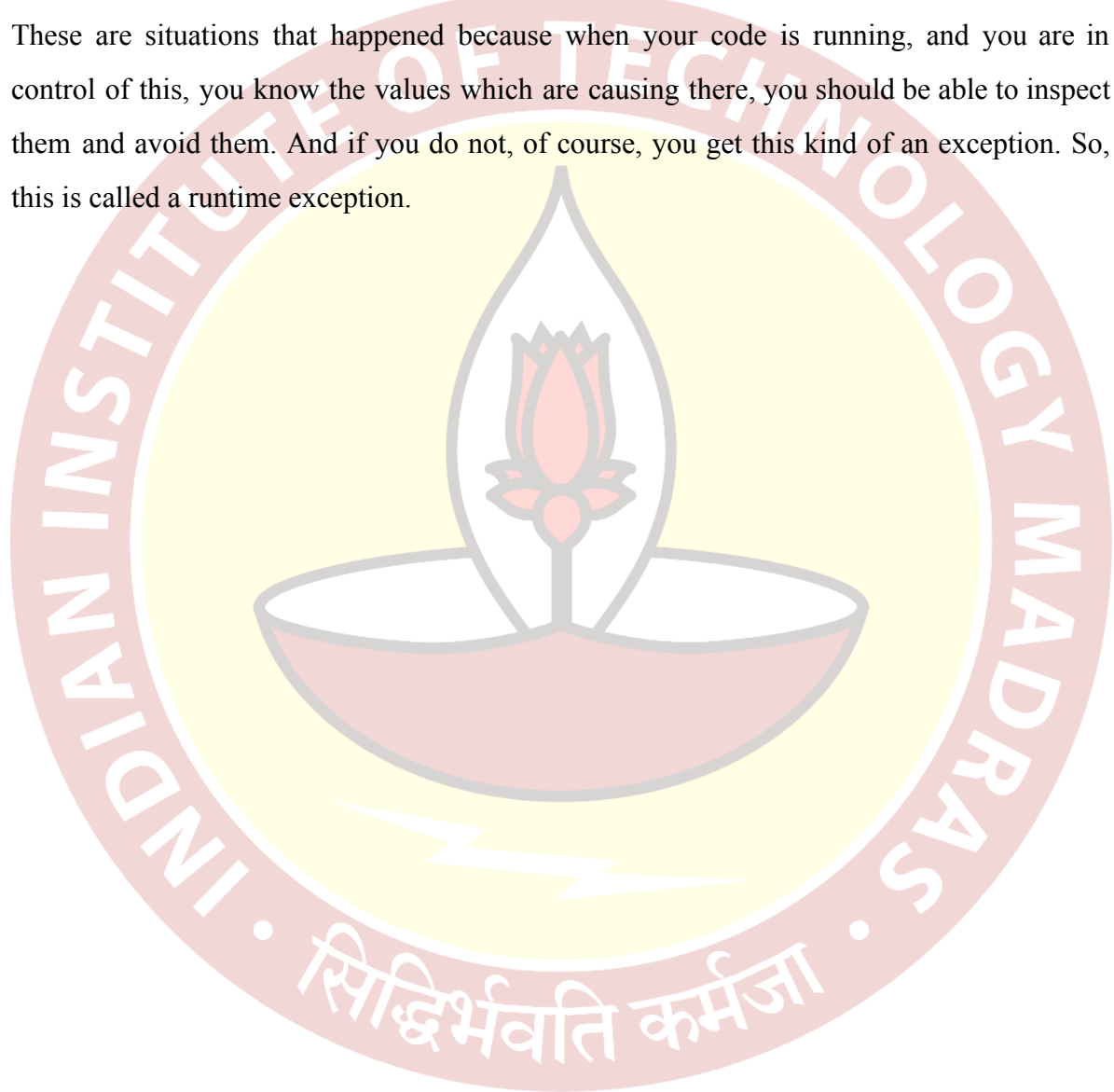
Handwritten notes on the second slide:
The text “RunTimeException — programming errors that should have been caught by code” is underlined. Below it, an example is given: “Array index out of bounds, invalid hash key, ...”. To the right, the notation $a[i]$ is written.

Now, on the other side, so remember that we had Throwable, and then we had error, and we had exception. So, exception itself splits into two things. So, there is a runtime exception, and there is everything else. So, everything else in Java terminology is called a checked exception. So, what is the runtime exception, a runtime exception is something that happens because your code goes wrong.

So, as opposed to error, which is not the programmer’s fault, in some sense, a runtime exception is the programmer’s fault, or at least is the programmer’s responsibility to check.

So, if you are accessing an element of an array, if you are doing `a[i]`, then you should check that `i` as a code programmer who wrote the code, you should check at this point that `i` is actually a legal value, you should not allow normally, the runtime to fail because of a runtime exception.

So, runtime exceptions are these kinds of coding errors, invalid hash keys, the things that we talked about before. And the main difference here is that these are artifacts of your code. These are situations that happened because when your code is running, and you are in control of this, you know the values which are causing there, you should be able to inspect them and avoid them. And if you do not, of course, you get this kind of an exception. So, this is called a runtime exception.



(Refer Slide Time: 12:40)



Java's classification of errors

- All exceptions descend from class `Throwable`
 - Two branches, `Error` and `Exception`
- `Error` — relatively rare, "not the programmer's fault"
 - Internal errors, resource limitations within Java runtime
 - No realistic corrective action possible, notify caller and terminate gracefully
- `Exception` — two sub branches
 - `RuntimeException`, `checked exceptions`
- `RuntimeException` — programming errors that should have been caught by code
 - Array index out of bounds, invalid hash key, ...
- Checked exceptions
 - Typically user-defined, code assumptions violated
 - In a list of orders, quantities should be positive integers



Madhavan Mukund

Dealing with errors

Programming Concepts using Java 4/5

And the more interesting kind of exception is the third one, which is the checked exception. So, this is typically when you want to signal certain errors, which are not as such errors with running the code, but they are errors related to the assumptions in your code. So, for instance, you might be keeping track of a list of orders with order numbers.

Now, order numbers should be integers. So, you would declare them to be integers. But Java does not have a built-in type of say, positive integers, you can only declare something to be of type integer. So, if you now create an `ArrayList`, or some other container, which has integers in it, then when you get a negative integer, what do you do?

So, one way is to flag it as a checked exception. So, you might say, I got a wrong thing. So, here is a negative integer exception. So, this is something which involves two things. One is it is an error check related to the code which is kind of subjective to that piece of code, it is not that every `ArrayList` is going to generate this error, unlike an index error out of bound in a normal array with any array will have to do because you cannot go outside the bounds of an array. Here, it is something specific to this array list, which keeps orders. And the second part is that the programmer has the flexibility to add this kind of an exception, so you can choose what kind of exception to report.

(Refer Slide Time: 14:00)



Summary

- Exception handling — gracefully recover from errors that occur when running code
- Throw an exception — generate an object encapsulating information about the error
- Catch an exception — decode the nature of the error and take corrective action
- Java organizes exceptions in a hierarchy, by type
 - **Error** — internal errors within JVM, "not the programmer's fault"
 - **RuntimeException** — coding errors, could have been avoided by runtime checks in code
 - Checked exceptions — user-defined, violations of assumptions made by code
 - To contrast, **Error** and **RuntimeException** are called unchecked exceptions.



Madhavan Mukund

Dealing with errors

Programming Concepts using Java 5/5

So, to summarize, we are going to look at exception handling. So, exception handling is the process of gracefully recovering from errors that occur when code is running. And it involves two parts. So, one part is the code generates the exception, so we can call it throwing an exception. So, of course in the underlying code may generate an exception because of a runtime error.

But you could also generate an exception, because the values that you see are not the values that you expect to see. So, these are exceptions that you might generate. So, when code recognizes an error has happened, it can flag it by throwing an exception and the flip side is the code that calls a function that could generate an error will be prepared to catch this exception and processing it, so it will decode the nature of the error and take some corrective action.

So, we saw that in Java, these exceptions are actually passed around as objects with structure so that you have information about the error that happened. So, these are structured objects are organized in the class hierarchy under parent class called Throwable. So, that the root class is called Throwable. And under Throwable, you have two subclasses error and exception, error deals with all these kinds of runtime errors which happen inside the JVM, which you cannot really do much about.

And under exception, you have runtime exception, which are coding errors, which the programmer should ideally keep track of, and try to avoid, or flag directly without letting

the underlying runtime crash. And the other side, you have these checked exceptions. So, checked exceptions are where you are actually looking for some logical error in the values based on the logic of the code.

And those are different types of exceptions, which is specific to a given application. So, because these are checked exceptions, both the other two the error and the runtime exception are sometimes called unchecked exceptions. So, you have so this is not technically part of the hierarchy because unchecked exceptions include two sides of that tree, the error side and the exceptions side. But to contrast them with checked exceptions.

